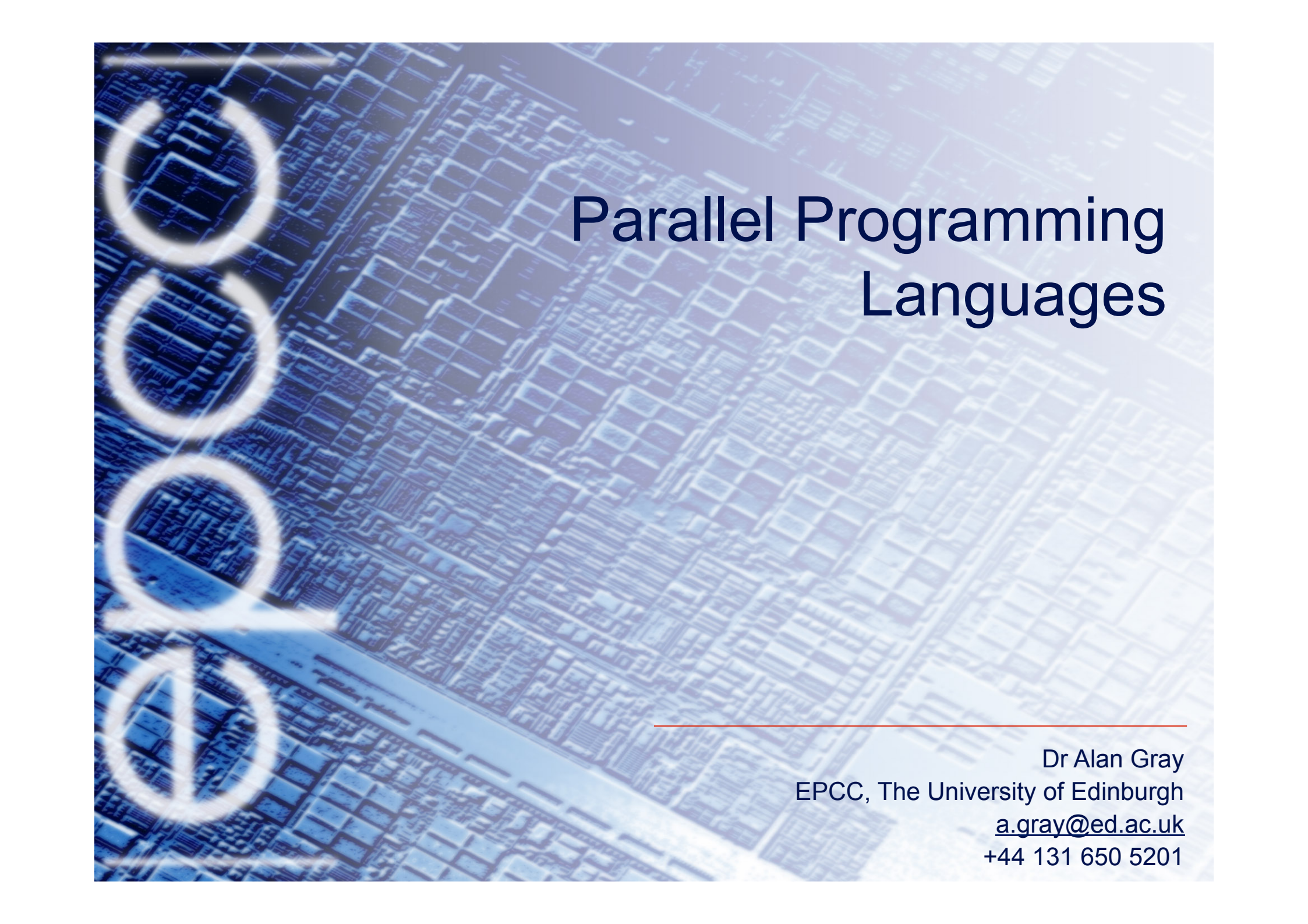




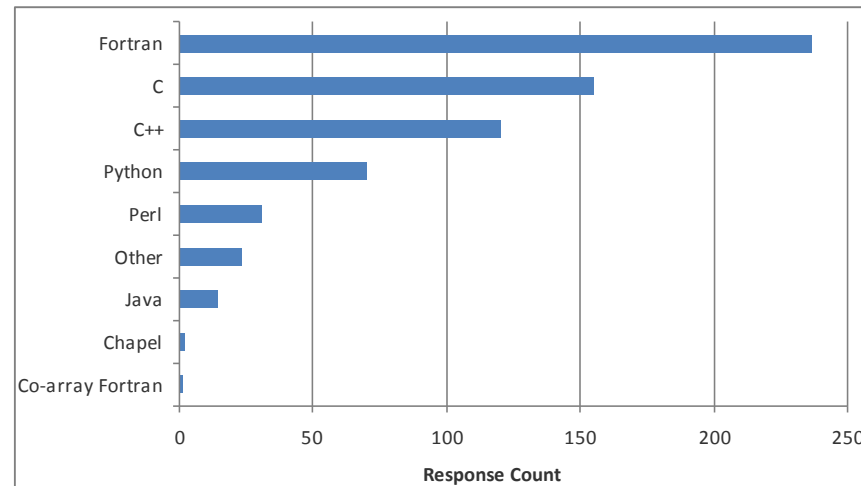
Parallel Programming Languages

Dr Alan Gray
EPCC, The University of Edinburgh
a.gray@ed.ac.uk
+44 131 650 5201

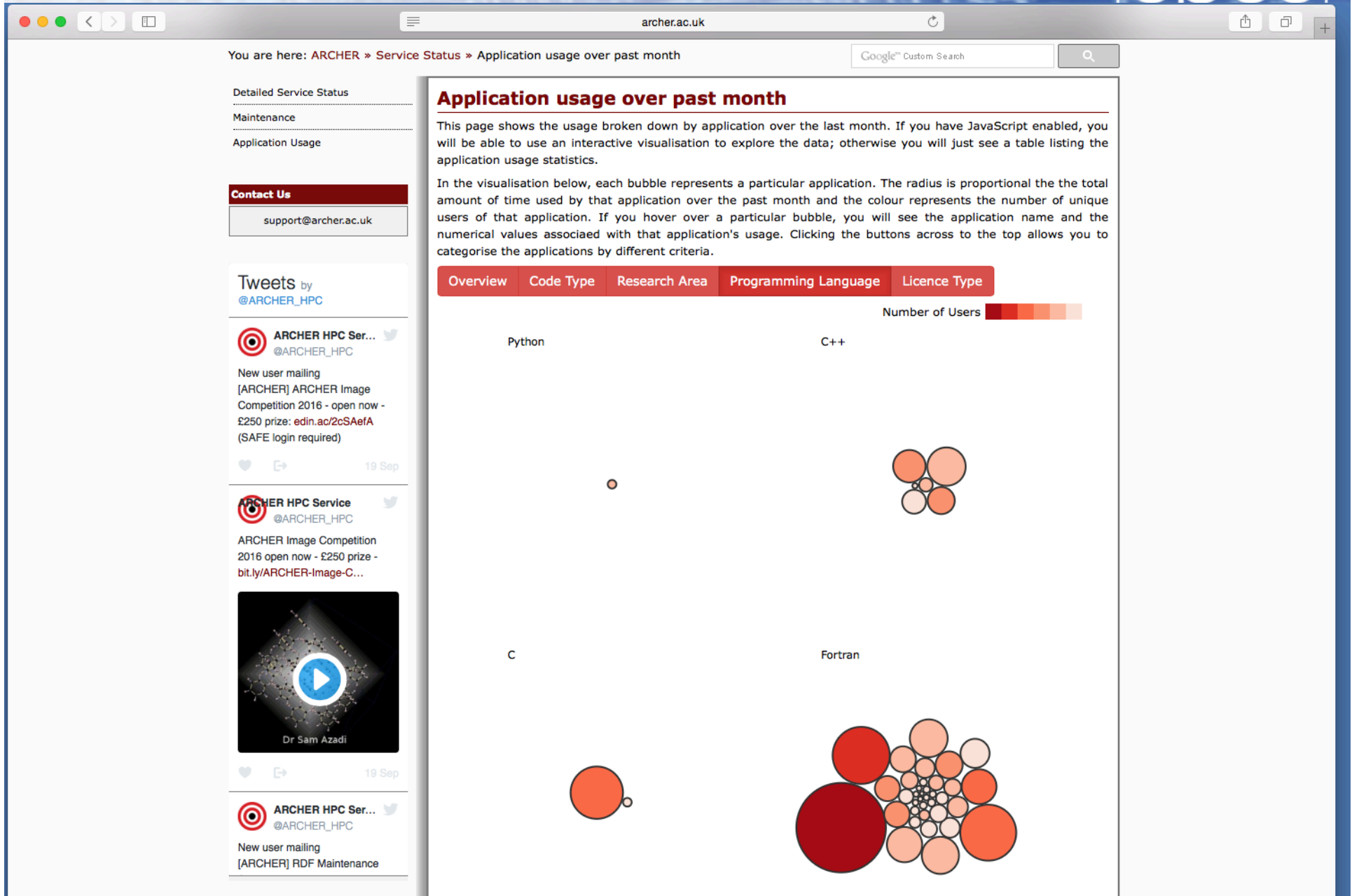
- Current Status of Programming Language Usage
 - Non-Parallel Programming Languages
 - Parallelisation Strategies
 - Mainstream HPC
- Alternative Parallel Programming Languages
 - Single-Sided Communication
 - PGAS
 - GPUs
 - Hybrid Approaches
- Final Remarks and Summary

Non-Parallel Programming Languages

- Serial (or base) languages form the basis of HPC applications
- PRACE Survey results:

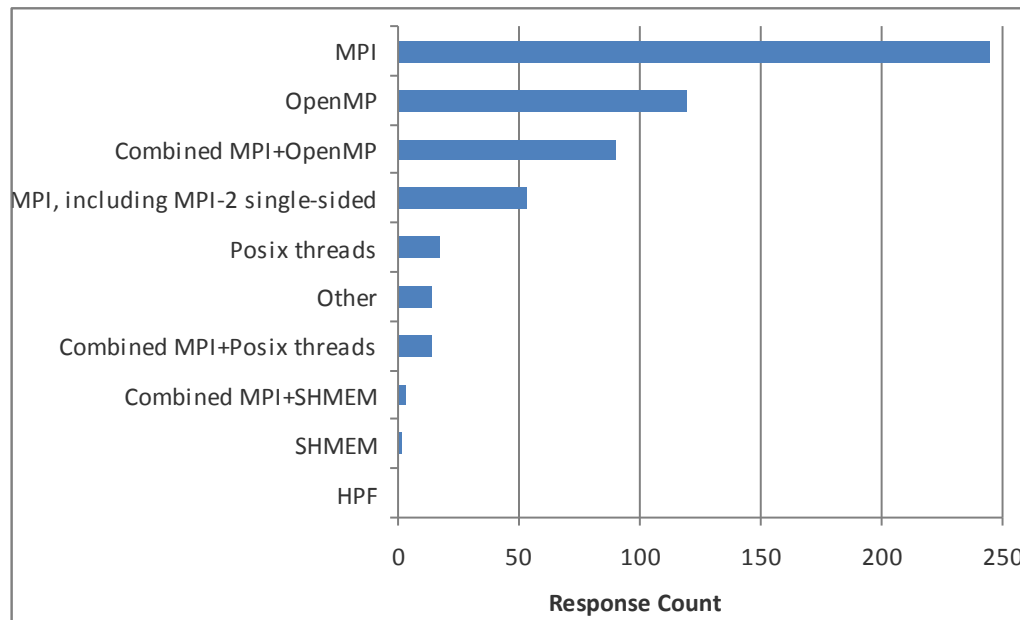


- PRACE Survey indicates that nearly all applications are written in:
 - Fortran: well suited for scientific computing
 - C/C++: allows good access to hardware
- Supplemented by
 - Scripts using Python, PERL and BASH
 - PGAS languages starting to be used



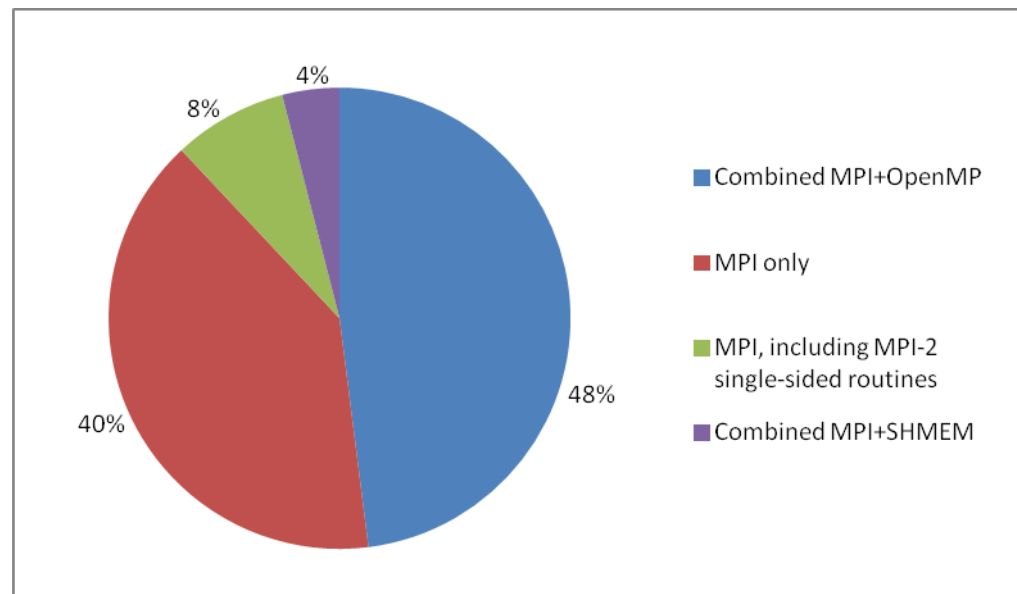
- Fortran is extremely popular as a base language for scientific computing
- You are now all expert Fortran programmers!
- Fortran also now has parallelism built in to the model
 - Co-Array Fortran (more later)
- C/C++ remain the other popular base languages
 - UPC is an extension of C with parallelism built in (more later)
- GPUs can be programmed using Fortran or C/C++ as a base language (again, more later)

- PRACE asked more than 400 European HPC users
 - “Which parallelisation implementations do you use?”



- Unsurprisingly, most popular answers were MPI, OpenMP and Combined MPI+OpenMP
 - Some users of Single-Sided communications

- PRACE also asked users of very largest systems:
 - “Which parallelisation method does your application use?”



- Most popular: “MPI Only” and “Combined MPI+OpenMP”
- 12% used single-sided routines

- For the last 15+years, most HPC cycles on large systems have been used to run MPI programs, written in Fortran or C/C++
 - Plus OpenMP used on shared memory systems/nodes
- MSc in HPC includes compulsory courses in MPI and OpenMP
- However, there are now reasons why this may be changing:
 - Currently, HPC systems have increasingly large numbers of cores, but the individual core performance is relatively static
 - There are new challenges in exploiting future Exascale systems
- So, alongside mainstream HPC, there is also significant activity in:
 - Single-sided communication
 - PGAS languages
 - Accelerators
 - Hybrid approaches
- *Many of these areas are discussed later in this course*

- *Multiple threads sharing global memory*
- Developed for systems with shared memory
- Program loop iterations can be distributed to threads
 - Each thread can refer to private objects within a parallel context
- Implementation
 - Threads map to user threads running on one shared memory node
 - Extensions to distributed memory not so successful
- Posix Threads/PThreads is a portable standard for threading
- Vendors had various shared-memory directives
- OpenMP developed as common standard for HPC
 - OpenMP is a good model to use within a node
 - More recent task features

- *Processes cooperate to solve problem by exchanging data*
- Can be used on most architectures
 - Especially suited for distributed memory systems
- The message passing model is based on the notion of *processes*
 - *Process*: an instance of a running program, together with the program's data
- Each process has access only to its own data
 - i.e., all variables are private
- Processes communicate with each other by sending+receiving messages
 - Typically library calls from a conventional sequential language
- During the 1980s, there was an explosion in message passing languages and libraries
 - CS Tools, OCCAM, CHIMP (developed by EPCC), PVM, PARMACS, ...

- *De facto* standard developed by working group of around 60 vendors and researchers from 40 organisations in USA and Europe
 - Took two years
 - MPI-1 released in 1993
 - Built on experiences from previous message passing libraries
- MPI's prime goals are:
 - To provide source-code portability
 - To allow efficient implementation
- MPI-2 and MPI-3 have subsequently been released, with a range of new features
 - Including single sided communications
- Now, MPI is used by nearly all message-passing programs

- *Allows direct access to memory of other processors*
 - Each process can access total memory, even on distributed memory systems
- Simpler protocol can bring performance benefits
 - But requires thinking about synchronisation, remote addresses, caching...
- Key routines
 - PUT is a remote write
 - GET is a remote read
- Functionality made available to application via library
- Vendor-specific libraries
 - SHMEM (Cray/SGI), LAPI (IBM)
- Portable implementations
 - MPI-2, OpenSHMEM

- Single-sided communication is major part of MPI-2 standard
 - Quite general and portable to most platforms
 - However, portability and robustness can have an impact on latency
 - Quite complicated and messy to use
- Better performance from lower-level interfaces, like SHMEM
 - Originally developed by Cray but a variety of similar implementations were developed on other platforms
 - Simple interface but hard to program correctly
- OpenSHMEM
 - Recent initiative to provide standard interface
 - See <http://www.openshmem.org>

- *Access to local memory via standard program mechanisms plus access to remote memory directly supported by language*
- The combination of access to all data plus also exploiting locality could give good performance and scaling
- Well suited to modern MIMD systems with multicore (shared memory) nodes
- Newly popular approach initially driven by US funding
 - Productive, Easy-to-use, Reliable Computing System (PERCS) project funded by DARPA's High Productivity Computing Systems (HPCS)

- Currently active and enthusiastic community
- Very wide variety of languages under the PGAS banner
 - See <http://www.pgas.org>
 - Including: CAF, UPC, Titanium, Fortress, X10, CAF 2.0, Chapel, Global Arrays, ...
- Often, these languages have more differences than similarities...

- The broad range of PGAS languages makes it difficult to choose which to use
- Currently, CAF and UPC are probably most relevant as Cray's compilers and hardware now support CAF and UPC in quite an efficient manner
- CAF: Fortran with Coarrays
 - Minimal addition to Fortran to support parallelism
 - Incorporated in Fortran 2008 standard!
- UPC: Unified Parallel C
 - Adding parallel features to C

- CPU cores are no longer getting faster
- Graphics Processing Units (GPUs) were originally developed for computer gaming and other graphical tasks, but for many years have been exploited for general purpose computing across a number of areas.
- They offer advantages over traditional CPUs because they have greater computational capability (many cores), and use high-bandwidth memory systems (where memory bandwidth is the main bottleneck for many scientific applications).





- Three “pre-exascale” machines have been announced in the US, each in the region of 100-300 petaflops
- *Summit* at ORNL and *Sierra* at LLNL will use **NVIDIA GPUs** (with IBM CPUs) .
- *Aurora* at Argonne will use **Intel Xeon Phi many-core CPUs** (Cray system)

- Graphics Processing Units (GPUs) have been increasing in performance much quicker than standard processor cores
- Led to an interest in GPGPU (General Purpose computation on Graphics Processing Units)
 - Where the GPU acts as an *accelerator* to the CPU
- NVIDIA GPUs are most common
- Variety of different ways to program GPUs
 - CUDA
 - NVIDIA's proprietary interface to the architecture.
 - Extensions to C (and Fortran) language which allow interfacing to the hardware
 - OpenCL
 - Cross platform API
 - Similar in design to CUDA, but lower level and not so mature
 - OpenACC
 - Directives-based approach
 - OpenMP-style directives - abstract complexities away from programmer

- *Use more than one parallelisation strategy within a single program*
- Trying to obtain more parallelisation by exploiting hierarchical parallelisation
- Most commonly, combining MPI + OpenMP
 - Using OpenMP across a shared-memory partition, with MPI to communicate between partitions
 - May make sense to use OpenMP just within a many-core processor
- But can also combine MPI with Pthreads or OpenSHMEM or CAF...
- Using many GPUs also often requires the use of MPI alongside the GPU programming approach

- It is not always entirely clear why some languages and approaches thrive while others fade away...
- However, languages which survive do have a number of common characteristics
 - Appropriate model for current hardware
 - Good portability
 - Ease of use
 - Applicable to a broad range of problems
 - Strong engagement from both vendors and user communities
 - Efficient implementations available

- Development of portable standards have been essential for uptake of new parallel programming ideas
- Mainstream HPC is currently based on MPI and OpenMP
 - However, there are alternatives
- Exascale challenges have injected new life into development of novel parallel programming languages and approaches
- The remainder of this course focuses on:
 - Co-Array Fortran
 - UPC
 - SHMEM
 - programming GPUs

- PRACE-PP
 - **D6.1: Identification and Categorisation of Applications and Initial Benchmarks Suite**, *Alan Simpson, Mark Bull and Jon Hill, EPCC*
- PRACE-1IP
 - **D7.4.1: Applications and user requirements for Tier-0 systems**, *Mark Bull (EPCC), Xu Guo (EPCC), Ioannis Liabotis (GRNET)*
 - **D7.4.3: Tier-0 Applications and Systems Usage**, *Xu Guo, Mark Bull (EPCC)*
- ARCHER User Requirements
 - Project Working Group: *Katharine Bowes (EPSRC), Ian Reid (NAG), Simon McIntosh-Smith (Bristol), Bryan Lawrence (NCAS/Reading) and Alan Simpson (EPCC)*
- *Acknowledgement: These slides have been adapted by a version written by Alan Simpson (the former PPL course organiser)*